DATA STRUCTURES

Interview Preparation Guide

■ JAVA EDITION — Beautified with Comments & Explanations

Arrays • Strings • Linked Lists • Stacks • Queues

Trees • Graphs • Hash Maps • Heaps • Tries • DP

Every code block is annotated with // WHY comments explaining the reasoning, data-structure choices, and algorithmic insights for each step.

■ JAVA COLLECTIONS — QUICK REFERENCE

Essential classes and when to use each one:

Java Collections Cheat-Sheet

```
// ■■■ ARRAY (fixed size, O(1) access) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
int[] arr = new int[n];
// WHY int[]: fastest option when size is known; no boxing overhead
// ■■■ DYNAMIC ARRAY (resizable, amortized O(1) append) ■■■■■■■■■■■■■■■■■■■■■■■■
List<Integer> list = new ArrayList<>();
// WHY ArrayList: use when you need dynamic size but still O(1) random access
// ■■■ STACK (LIFO — Last In First Out) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Deque<Integer> stack = new ArrayDeque<>();
stack.push(x); // add to top O(1)
stack.pop(); // remove top O(1)
stack.peek(); // view top O(1)
// WHY ArrayDeque over Stack: ArrayDeque is faster (no synchronization overhead)
// ■■■ QUEUE (FIFO — First In First Out) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Queue<Integer> queue = new LinkedList<>();
queue.offer(x); // enqueue O(1)
queue.poll(); // dequeue O(1)
queue.peek(); // front O(1)
// WHY Queue for BFS: processes nodes level by level (breadth-first)
// ■■■ DEQUE (double-ended — both Stack + Queue) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Deque<Integer> dq = new ArrayDeque<>();
dq.offerFirst(x); dq.offerLast(x);
dq.pollFirst(); dq.pollLast();
// WHY Deque: sliding window max problems need add/remove from both ends
// ■■■ MIN-HEAP (smallest element always at root) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
// WHY PriorityQueue: O(log n) insert/extract — perfect for Top-K problems
// ■■■ MAX-HEAP (largest element always at root) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());
// ■■■ CUSTOM HEAP (e.g. sort int[] by first element) ■■■■■■■■■■■■■■■■■■■■■■■■■■
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
// WHY lambda comparator: a[0]-b[0] = ascending (min-heap); b[0]-a[0] = descending
// ■■■ HASHMAP (O(1) avg for get/put/containsKey) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Map<Integer, Integer> map = new HashMap<>();
map.getOrDefault(key, 0); // safe read — avoids NullPointerException
map.merge(key, 1, Integer::sum); // increment count in one line
map.computeIfAbsent(key, k -> new ArrayList<>()).add(val); // group items
// WHY HashMap: frequency counting, two-sum lookup, memoization
// ■■■ TREEMAP (sorted by key, O(log n)) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
TreeMap<Integer, Integer> tm = new TreeMap<>();
tm.floorKey(k); // largest key <= k
tm.ceilingKey(k); // smallest key >= k
// WHY TreeMap: when you need sorted order + range queries (e.g. time-based store)
// ■■■ LINKEDHASHMAP (insertion-order OR access-order) ■■■■■■■■■■■■■■■■■■■■■■■■■
Map<Integer, Integer> lhm = new LinkedHashMap<>(16, 0.75f, true);
// WHY accessOrder=true: moves recently used entry to tail -> perfect for LRU Cache
```

1. ARRAYS

Core Concepts

- Contiguous memory — O(1) random access by index, O(n) insert/delete
- int[] in Java is fixed size. Use ArrayList<Integer> for dynamic resizing
- Arrays.sort(arr) — O(n log n). For primitives: Dual-Pivot QuickSort. For objects: TimSort (stable)
- Most array interview problems reduce to: Two Pointers, Sliding Window, or Prefix Sum

Two Pointers — Two Sum on Sorted Array

```
// PATTERN: Two Pointers — O(n) time, O(1) space
// PROBLEM: Given sorted array, find two numbers that sum to target
// WHY Two Pointers: sorting gives us order; we exploit it to skip O(n^2) brute force
public int[] twoSumSorted(int[] arr, int target) {
// STEP 1: Place one pointer at start, one at end
// WHY: If sum is too small, move left pointer right (increase sum)
// If sum is too big, move right pointer left (decrease sum)
int left = 0;
int right = arr.length - 1;
while (left < right) {
int sum = arr[left] + arr[right];
if (sum == target) {
return new int[]{left, right}; // found it
} else if (sum < target) {
left++; // WHY: need bigger sum, advance left
} else {
right--; // WHY: need smaller sum, retreat right
}
}
return new int[]{-1, -1}; // no pair found
}
// TIME: O(n) — single pass SPACE: O(1) — no extra memory
```

■ WHY THIS WORKS: Sorting creates monotone structure. Two pointers exploit it — moving left increases sum, moving right decreases it. This turns O(n²) brute force into O(n).

Sliding Window — Max Sum Subarray of Size K

```
// PATTERN: Sliding Window (Fixed Size) — O(n) time, O(1) space
// PROBLEM: Find maximum sum subarray of exactly size k
// WHY Sliding Window: instead of recalculating sum from scratch each time,
// we slide the window — subtract the element leaving, add element entering
public int maxSumSubarray(int[] arr, int k) {
// STEP 1: Compute sum of first window [0 .. k-1]
int windowSum = 0;
for (int i = 0; i < k; i++) {
windowSum += arr[i];
}
int best = windowSum;
// STEP 2: Slide window one position at a time
// WHY: arr[i] enters the window, arr[i-k] leaves the window
for (int i = k; i < arr.length; i++) {
```

```
windowSum += arr[i]; // new element enters from right
windowSum -= arr[i - k]; // old element leaves from left
best = Math.max(best, windowSum);
}
return best;
}
// TIME: O(n) — two passes SPACE: O(1)
```

■ WHY THIS WORKS: Naive approach: O(n*k) — recalculate full sum each time. Sliding window reuses previous sum: O(n). We only process each element once (in + out).

Prefix Sum + HashMap — Subarray Sum Equals K

```
// PATTERN: Prefix Sum + HashMap — O(n) time, O(n) space
// PROBLEM: Count subarrays with sum exactly equal to k
// WHY: We can't use sliding window (negative numbers possible)
// Instead, prefix sum trick: if prefixSum[j] - prefixSum[i] == k
// then subarray [i+1..j] sums to k
public int subarraySum(int[] nums, int k) {
// STEP 1: HashMap stores (prefixSum -> how many times we've seen it)
// WHY: We need to count, not just detect, so we store frequency
Map<Integer, Integer> freq = new HashMap<>();
freq.put(0, 1); // WHY put(0,1): handles subarrays starting at index 0
int count = 0;
int prefixSum = 0;
for (int n : nums) {
prefixSum += n;
// STEP 2: Check if (prefixSum - k) was seen before
// WHY: If prefixSum - k exists, the subarray between those points sums to k
count += freq.getOrDefault(prefixSum - k, 0);
// STEP 3: Record current prefixSum in map
freq.merge(prefixSum, 1, Integer::sum);
}
return count;
}
// TIME: O(n) SPACE: O(n) for the HashMap
```

■ WHY THIS WORKS: Key insight: sum(i..j) = prefix[j] - prefix[i-1]. So if prefix[j] - k appeared before as a prefix, that subarray sums to k. HashMap gives O(1) lookup.

Kadane's Algorithm — Maximum Subarray Sum

```
// PATTERN: Kadane's Algorithm — O(n) time, O(1) space
// PROBLEM: Find the contiguous subarray with the largest sum
// WHY Kadane's: At each position, we make a greedy choice:
// - Extend the current subarray (add current element)
// - OR start a fresh subarray from current element
// Whichever gives more is kept
public int maxSubArray(int[] nums) {
// STEP 1: Start with first element as both current and global max
int maxCurrent = nums[0]; // best subarray ending exactly at current index
int maxGlobal = nums[0]; // best subarray seen anywhere so far
for (int i = 1; i < nums.length; i++) {
// STEP 2: Greedy choice — extend existing or start fresh?
```

```
// WHY Math.max: if maxCurrent went negative, it HURTS to extend it
// starting fresh gives nums[i] alone, which is better
maxCurrent = Math.max(nums[i], maxCurrent + nums[i]);
// STEP 3: Update global maximum if current subarray is best so far
maxGlobal = Math.max(maxGlobal, maxCurrent);
}
return maxGlobal;
}
// TIME: O(n) SPACE: O(1) — only 2 variables, no array needed
```

■ WHY THIS WORKS: Greedy insight: a negative prefix sum always hurts — discard it and restart. maxCurrent tracks best ending here; maxGlobal is the answer.

Top Array Interview Questions

Two Sum (unsorted — HashMap)

Use map to store value→index. For each num, check if target-num is in map. O(n) time, O(n) space

Container With Most Water

Two pointers. area = min(height[l], height[r]) * (r - l). Move the shorter side inward

3Sum

Sort array. Fix index i, then two-pointer on [i+1, n-1]. Skip duplicates carefully

Product of Array Except Self

Left pass: product of all elements to the left. Right pass: multiply by product to the right. No division needed

Trapping Rain Water

For each cell: water = min(maxLeft, maxRight) - height[i]. Precompute leftMax[] and rightMax[] arrays

Jump Game

Greedy. Track maxReach. If i > maxReach at any point, return false

Merge Intervals

Sort by start time. For each interval, if it overlaps with last in result, extend end. Else add new interval

Rotate Array by K

Reverse all → reverse first k → reverse rest. O(n) time, O(1) space

Find Duplicate (Floyd's Cycle)

Treat array as linked list (val = next node). Detect cycle with slow/fast pointers

Maximum Product Subarray

Track both maxProd and minProd (negatives can flip sign). Update at each step

2. STRINGS

Core Concepts in Java

- String is IMMUTABLE — every + creates a new object. Use StringBuilder for building
- s.charAt(i) not s[i] | s.length() not s.length | s.toCharArray() for char manipulation
- int[26] count array is fastest for lowercase letter frequency (no HashMap overhead)
- Character utility: Character.isLetterOrDigit(c) Character.toLowerCase(c)

Sliding Window — Longest Substring Without Repeating Characters

```
// PATTERN: Sliding Window (Variable Size) — O(n) time, O(1) space (26-char map)
// PROBLEM: Find the length of the longest substring without repeating characters
// WHY Sliding Window: we grow the window right, shrink from left when duplicate found
// The key insight: we only ever MOVE FORWARD — no nested loops needed
public int lengthOfLongestSubstring(String s) {
// STEP 1: Map stores char -> its most recent index
// WHY HashMap not array: handles any Unicode char, not just lowercase
Map<Character, Integer> seen = new HashMap<>();
int left = 0; // left boundary of current valid window
int best = 0;
for (int right = 0; right < s.length(); right++) {
char c = s.charAt(right);
// STEP 2: If char was seen AND is still inside our current window
// WHY seen.get(c) >= left: char might have appeared before the window — ignore it
if (seen.containsKey(c) && seen.get(c) >= left) {
left = seen.get(c) + 1; // shrink window from left
}
// STEP 3: Record/update most recent position of this char
seen.put(c, right);
// STEP 4: Update answer — window size is right - left + 1
best = Math.max(best, right - left + 1);
}
return best;
}
// TIME: O(n) — each char processed once SPACE: O(min(n, alphabet))
```

■ **WHY THIS WORKS: Key insight: when a duplicate is found, we don't restart from scratch. We jump left pointer just past the previous occurrence. Each character is visited at most twice — O(n) total.**

Frequency Array — Valid Anagram

```
// PATTERN: Frequency Array — O(n) time, O(1) space
// PROBLEM: Check if two strings are anagrams of each other
// WHY int[26]: anagram means same chars same counts. int array is faster than HashMap
// because array access is O(1) with no hashing overhead
public boolean isAnagram(String s, String t) {
// STEP 1: Length check — different lengths can't be anagrams
if (s.length() != t.length()) return false;
// STEP 2: Single frequency array — increment for s, decrement for t
// WHY one array: equivalent to comparing two separate count arrays but O(1) space
int[] count = new int[26]; // slots 0-25 for 'a'-'z'
for (char c : s.toCharArray()) count[c - 'a']++; // +1 for each char in s
```

```
for (char c : t.toCharArray()) count[c - 'a']--; // -1 for each char in t
// STEP 3: If all counts are 0, every char balanced out — perfect anagram
for (int x : count) {
if (x != 0) return false; // imbalance means chars don't match
}
return true;
}
// TIME: O(n) SPACE: O(1) — array size fixed at 26, independent of input
```

Two Pointers — Valid Palindrome

```
// PATTERN: Two Pointers + char filtering — O(n) time, O(1) space
// PROBLEM: Check if a string is a palindrome (ignore non-alphanumeric, case-insensitive)
public boolean isPalindrome(String s) {
int left = 0;
int right = s.length() - 1;
while (left < right) {
// STEP 1: Skip non-alphanumeric chars on both sides
// WHY: Problem says "only consider alphanumeric characters"
while (left < right && !Character.isLetterOrDigit(s.charAt(left))) left++;
while (left < right && !Character.isLetterOrDigit(s.charAt(right))) right--;
// STEP 2: Compare chars (case-insensitive)
if (Character.toLowerCase(s.charAt(left)) !=
Character.toLowerCase(s.charAt(right))) {
return false; // mismatch found — not a palindrome
}
left++;
right--;
}
return true; // all pairs matched
}
// TIME: O(n) SPACE: O(1) — no extra data structures
```

HashMap Grouping — Group Anagrams

```
// PATTERN: Sorting + HashMap grouping — O(n * k log k) where k = max word length
// PROBLEM: Group strings that are anagrams of each other
// WHY sorting: two strings are anagrams iff their sorted forms are identical
// Sorted form becomes the "canonical key" for grouping
public List<List<String>> groupAnagrams(String[] strs) {
// STEP 1: Map from canonical key -> list of words with that key
// WHY computeIfAbsent: creates list on first encounter, avoids null check
Map<String, List<String>> groups = new HashMap<>();
for (String word : strs) {
// STEP 2: Sort the characters to get the canonical form
char[] chars = word.toCharArray();
Arrays.sort(chars); // "eat","tea","ate" all -> "aet"
String key = new String(chars); // WHY new String: convert array back to string
// STEP 3: Add word to its group (create group if first time)
groups.computeIfAbsent(key, k -> new ArrayList<>()).add(word);
}
return new ArrayList<>(groups.values());
}
// TIME: O(n * k log k) SPACE: O(n * k) for the map
```

Top String Interview Questions

Minimum Window Substring

Sliding window. need=freq of target chars. when have==need, shrink from left while valid. Track min window

Longest Palindromic Substring

Expand from center. Two cases: odd length (center=i) and even (center=i,i+1). Track best start+length

Palindromic Substrings (count)

Same expand-from-center. Count each valid palindrome during expansion

Word Break (DP)

boolean[] dp where dp[i]=true if s[0..i-1] can be formed. Use HashSet for O(1) word lookup

Valid Parentheses

Deque as stack. Push open brackets. On close: if stack empty or top doesn't match → false

Encode & Decode Strings

Length-prefix encoding: '4#word'. Encode: len+'#'+str. Decode: scan for '#', extract length

Longest Repeating Char Replacement

Sliding window. Key: (windowSize - maxFreqChar) <= k. If not, shrink from left

Reverse Words in String

s.trim().split('\\s+') then reverse array then String.join(' ', parts)

3. LINKED LISTS

ListNode Definition & Setup Patterns

```
// NODE DEFINITION — memorize this for every linked list problem
class ListNode {
int val; // the data stored in this node
ListNode next; // pointer to the next node (null if last)
ListNode(int val) { this.val = val; }
}
// COMMON SETUP PATTERNS:
// WHY dummy head: many ops change head (insert before, remove head)
// dummy.next = head avoids special-casing the head node
ListNode dummy = new ListNode(0);
dummy.next = head;
ListNode curr = dummy;
// WHY two-pointer naming: descriptive names prevent pointer confusion
ListNode slow = head; // moves 1 step at a time
ListNode fast = head; // moves 2 steps at a time
```

In-place Reversal — Reverse Linked List

```
// PATTERN: In-place Iterative Reversal — O(n) time, O(1) space
// PROBLEM: Reverse a singly linked list
// WHY iterative over recursive: recursive uses O(n) call stack space
// KEY INSIGHT: we must save next before overwriting the .next pointer
public ListNode reverseList(ListNode head) {
ListNode prev = null; // will become the new tail (points to null)
ListNode curr = head; // starts at original head
// BEFORE: null <- ? ... prev curr -> next -> ...
// AFTER: null <- [reversed chain] prev curr=null
while (curr != null) {
// STEP 1: Save next BEFORE we overwrite curr.next
// WHY: once we set curr.next = prev, we'd lose reference to the rest
ListNode nextNode = curr.next;
// STEP 2: Reverse the pointer
curr.next = prev; // point current node backward
// STEP 3: Advance both pointers one step forward
prev = curr; // prev moves to current node
curr = nextNode; // curr moves to saved next
}
// After loop: curr == null, prev == new head
return prev;
}
// TIME: O(n) SPACE: O(1) — only 3 pointer variables
```

■ **WHY THIS WORKS: Three-pointer dance: nextNode saves the link we're about to break. prev accumulates the reversed chain. curr advances through the original. No extra memory needed.**

Floyd's Algorithm — Detect Cycle in Linked List

```
// PATTERN: Fast & Slow Pointers (Floyd's Cycle Detection) — O(n) time, O(1) space
// PROBLEM: Detect if a linked list has a cycle
// WHY fast/slow: if there IS a cycle, fast will eventually lap slow
```

```
// Think of it like two runners on a circular track — faster always catches slower
public boolean hasCycle(ListNode head) {
ListNode slow = head; // moves 1 step per iteration
ListNode fast = head; // moves 2 steps per iteration
while (fast != null && fast.next != null) {
// WHY check fast.next too: fast moves 2 steps, so next must exist
slow = slow.next; // advance 1
fast = fast.next.next; // advance 2
// If pointers meet, there's a cycle
// WHY they meet: in a cycle of length C, fast gains 1 step per iteration
// so they meet after at most C iterations
if (slow == fast) return true;
}
// fast hit null -> no cycle (list terminates)
return false;
}
// TIME: O(n) SPACE: O(1) — no HashSet needed (unlike naive approach)
```

■ **WHY THIS WORKS: Naive: use HashSet to record visited nodes — O(n) space. Floyd's: mathematical insight that in a cycle, fast pointer ALWAYS catches slow. O(1) space.**

Dummy Head + Merge — Merge Two Sorted Lists

```
// PATTERN: Dummy Head + Merge — O(n+m) time, O(1) space
// PROBLEM: Merge two sorted linked lists into one sorted list
// WHY dummy head: the merged list's head is unknown initially
// dummy.next will be our answer — avoids null checks for empty list
public ListNode mergeTwoLists(ListNode l1, ListNode l2) {
// STEP 1: Dummy head — gives us a stable starting point
ListNode dummy = new ListNode(0);
ListNode curr = dummy; // curr builds the merged list
// STEP 2: While both lists have nodes, pick the smaller one
// WHY compare val: both are sorted, so smaller head is next in merged order
while (l1 != null && l2 != null) {
if (l1.val <= l2.val) {
curr.next = l1; // attach l1 node
l1 = l1.next; // advance l1
} else {
curr.next = l2; // attach l2 node
l2 = l2.next; // advance l2
}
curr = curr.next; // advance merged list pointer
}
// STEP 3: Attach remaining nodes (at most one list is non-empty)
// WHY ternary: avoids if-else; one of them is null, other is remaining chain
curr.next = (l1 != null) ? l1 : l2;
return dummy.next; // skip the dummy — return actual head
}
// TIME: O(n + m) SPACE: O(1) — no new nodes created, just pointer rewiring
```

LRU Cache — LinkedHashMap with Access-Order

```
// PATTERN: LinkedHashMap LRU Cache — O(1) get and put
// PROBLEM: Design a cache that evicts the Least Recently Used item when full
```

```
// WHY LinkedHashMap(accessOrder=true): automatically moves accessed entries to tail
// Eldest (head) = least recently used = evict this one
class LRUCache extends LinkedHashMap<Integer, Integer> {
private final int capacity;
// STEP 1: Constructor — enable access-order mode
// WHY accessOrder=true: on every get() or put(), that entry moves to the END
// So the FRONT is always the Least Recently Used
public LRUCache(int capacity) {
super(capacity, 0.75f, true); // initialCap, loadFactor, accessOrder
this.capacity = capacity;
}
public int get(int key) {
// LinkedHashMap.get() already moves key to tail (most-recent position)
return super.getOrDefault(key, -1);
}
public void put(int key, int value) {
// LinkedHashMap.put() also moves key to tail
super.put(key, value);
}
// STEP 2: Override to auto-evict when over capacity
// WHY removeEldestEntry: called after every put(); eldest = LRU = head
@Override
protected boolean removeEldestEntry(Map.Entry<Integer, Integer> eldest) {
return size() > capacity; // true means: please remove eldest
}
}
// TIME: O(1) for both get() and put() SPACE: O(capacity)
```

■ **WHY THIS WORKS: LinkedHashMap maintains a doubly-linked list internally. With accessOrder=true, every access moves that node to the tail. The head is always the LRU item. removeEldestEntry auto-evicts it when we exceed capacity.**

Top Linked List Interview Questions

Find Middle of Linked List

Fast/slow: when fast reaches end, slow is at middle. For even-length, slow lands at second middle

Remove Nth Node from End

Two pointers n apart. Move both until first hits null. Second is at (n+1)th from end — remove its next

Merge K Sorted Lists

PriorityQueue<ListNode> sorted by val. Poll min node, advance that list, offer next node

Reorder List

1) Find middle 2) Reverse second half 3) Merge two halves alternately

Palindrome Linked List

Find middle, reverse second half, compare both halves node-by-node

Copy List with Random Pointer

HashMap<Node,Node> old→new. Two passes: first create all nodes, then set .next and .random

Add Two Numbers

Dummy head + carry variable. Handle lists of different lengths + carry overflow

Sort List

Merge sort: findMiddle, split, sortList(left), sortList(right), merge. O(n log n) time O(log n) space

Find Cycle Entry Point

After slow==fast, reset one pointer to head. Advance both 1 step — they meet at cycle entry

Intersection of Two Lists

Two pointers switch to other head on null. They meet after traversing equal total distance

4. STACKS & QUEUES

Stack — Valid Parentheses

```
// PATTERN: Stack (Deque) for bracket matching — O(n) time, O(n) space
// PROBLEM: Determine if string of brackets is valid
// WHY Stack: brackets must close in LIFO order — last opened must close first
// '(' pushed; when ')' seen, it must match the most-recently-pushed '('
public boolean isValid(String s) {
// WHY ArrayDeque: faster than Stack class (no synchronization)
Deque<Character> stack = new ArrayDeque<>();
for (char c : s.toCharArray()) {
// STEP 1: Push opening brackets onto stack
if (c == '(' || c == '[' || c == '{') {
stack.push(c);
// STEP 2: For closing brackets, stack top MUST be matching open bracket
} else {
// WHY isEmpty() check first: prevents EmptyDequeException
if (stack.isEmpty()) return false;
char top = stack.pop();
// Verify pair matches
if (c == ')' && top != '(') return false;
if (c == ']' && top != '[') return false;
if (c == '}' && top != '{') return false;
}
}
// STEP 3: Stack must be empty — all brackets closed
// WHY: unclosed brackets remain on stack
return stack.isEmpty();
}
// TIME: O(n) SPACE: O(n) worst case (all opening brackets)
```

■ WHY THIS WORKS: Stack naturally models LIFO bracket pairing. Each ')' must match its most recent '('. If the stack top doesn't match, the sequence is invalid. Empty stack at end = all brackets closed.

Monotonic Stack — Daily Temperatures

```
// PATTERN: Monotonic Stack (decreasing) — O(n) time, O(n) space
// PROBLEM: For each element, find the next element that is greater than it
// WHY Monotonic Stack: maintain elements in decreasing order
// When a larger element arrives, it IS the "next greater" for all smaller elements
// currently in the stack — pop them and record their answer
public int[] dailyTemperatures(int[] temps) {
int n = temps.length;
int[] result = new int[n]; // default 0 = no warmer day
Deque<Integer> stack = new ArrayDeque<>(); // stores INDICES (not values)
// WHY store indices: we need to calculate distances (j - i)
for (int i = 0; i < n; i++) {
// STEP 1: While stack not empty AND current temp > temp at stack's top index
// WHY pop: current day is the "first warmer day" for the popped day
while (!stack.isEmpty() && temps[i] > temps[stack.peek()]) {
int prevDay = stack.pop();
result[prevDay] = i - prevDay; // days to wait = current index - that day's index
```

```
}
// STEP 2: Push current index — we don't know its answer yet
stack.push(i);
}
// Any index left in stack never finds a warmer day → remains 0
return result;
}
// TIME: O(n) — each element pushed and popped at most once SPACE: O(n)
```

■ WHY THIS WORKS: Monotonic stack amortizes cost: each element is pushed ONCE and popped ONCE — O(n) total, not O(n²). Stack always holds 'unsettled' days waiting for their warmer answer.

Monotonic Deque — Sliding Window Maximum

```
// PATTERN: Monotonic Deque for Sliding Window — O(n) time, O(k) space
// PROBLEM: Find maximum in every sliding window of size k
// WHY Deque: we need to add/remove from BOTH ends
// - Remove from FRONT: when window moves past an old index
// - Remove from BACK: when new element is bigger (old smaller elements useless)
public int[] maxSlidingWindow(int[] nums, int k) {
int n = nums.length;
int[] result = new int[n - k + 1];
int ri = 0;
// STEP 1: Deque stores INDICES in decreasing order of their VALUES
// WHY indices not values: we need to check if index is still within window
Deque<Integer> dq = new ArrayDeque<>();
for (int i = 0; i < n; i++) {
// STEP 2: Remove indices no longer in current window from FRONT
// WHY: window is [i-k+1, i]; anything older is out of scope
while (!dq.isEmpty() && dq.peekFirst() < i - k + 1) {
dq.pollFirst();
}
// STEP 3: Remove from BACK any index with smaller value than nums[i]
// WHY: those elements can NEVER be the max — nums[i] is bigger AND newer
// They won't be the answer for any future window either
while (!dq.isEmpty() && nums[dq.peekLast()] < nums[i]) {
dq.pollLast();
}
dq.offerLast(i); // add current index to back
// STEP 4: Once we have a full window, record max (front of deque)
// WHY front is max: deque is in decreasing order, front = largest
if (i >= k - 1) {
result[ri++] = nums[dq.peekFirst()];
}
}
return result;
}
// TIME: O(n) — each element added and removed from deque at most once
```

Auxiliary Stack — Min Stack

```
// PATTERN: Auxiliary Min-Stack — O(1) for all operations
// PROBLEM: Design a stack that also supports getMin() in O(1)
// WHY second stack: regular stack loses track of previous minimums on pop()
// Parallel minStack mirrors main stack, storing the current min AT EACH LEVEL
```

```
class MinStack {
// WHY two stacks: main holds actual values; minStack holds running minimum
Deque<Integer> stack = new ArrayDeque<>();
Deque<Integer> minStack = new ArrayDeque<>();
public void push(int val) {
stack.push(val);
// STEP: Push current minimum to minStack
// WHY: if val is smaller, update min; otherwise carry forward the same min
int currentMin = minStack.isEmpty() ? val : Math.min(val, minStack.peek());
minStack.push(currentMin);
}
public void pop() {
stack.pop();
minStack.pop(); // WHY: pop both to keep them synchronized
}
public int top() { return stack.peek(); }
public int getMin() { return minStack.peek(); }
// WHY O(1): minStack always has the answer at its top — no scanning needed
}
// TIME: O(1) all operations SPACE: O(n) — minStack mirrors main stack
```

■ WHY THIS WORKS: The trick: at every push, we record what the minimum IS at that stack depth. On pop, both stacks shrink together, restoring the previous minimum instantly.

Top Stack & Queue Interview Questions

Evaluate Reverse Polish Notation

Stack: if number push it; if operator pop two, compute, push result

Implement Queue Using Two Stacks

inStack for push. On poll/peek, if outStack empty: transfer all from inStack. Amortized O(1)

Largest Rectangle in Histogram

Monotonic increasing stack of indices. On smaller bar, pop and calculate area = h * (i - stack.peek() - 1)

Basic Calculator I/II

Stack for numbers + signs. Handle spaces, +/- with sign variable, */ directly

Decode String

Stack for (repeatCount, builtString). On ']', pop and repeat current string

Asteroid Collision

Stack. Positive: push. Negative: resolve collisions with stack top until stable

Remove K Digits

Monotonic increasing stack. Pop when next digit is smaller. Remove remaining from top

Design Circular Queue

int[] with front, rear, size. (rear+1) % capacity to wrap around

Number of Recent Calls

ArrayDeque queue. Offer t. Poll all < t-3000. Size = answer

Trapping Rain Water (Stack)

Monotonic stack. On taller bar: pop, calculate water using height diff and width

5. TREES

TreeNode Definition & Traversal Overview

```
// NODE DEFINITION — standard for all tree problems
class TreeNode {
int val;
TreeNode left; // left subtree (values smaller in BST)
TreeNode right; // right subtree (values larger in BST)
TreeNode(int val) { this.val = val; }
}
// TRAVERSAL TEMPLATES (memorize all three)
// Inorder (Left → Root → Right) → gives SORTED sequence for BST
// Preorder (Root → Left → Right) → used for serialization / copy
// Postorder (Left → Right → Root) → used for deletion / bottom-up info
```

Recursive DFS — Maximum Depth of Binary Tree

```
// PATTERN: Recursive DFS (Post-order) — O(n) time, O(h) space (h = height)
// PROBLEM: Find the maximum depth (height) of a binary tree
// WHY Recursion: tree structure is naturally recursive — each subtree is a tree
// WHY Post-order: we need children's depths BEFORE we can compute parent's depth
public int maxDepth(TreeNode root) {
// BASE CASE: null node contributes 0 to depth
// WHY: recursion must stop when we fall off a leaf
if (root == null) return 0;
// STEP 1: Recursively get depth of left and right subtrees
int leftDepth = maxDepth(root.left);
int rightDepth = maxDepth(root.right);
// STEP 2: Current node adds 1 to the deeper of the two subtrees
// WHY Math.max: we take the LONGEST path to any leaf
return 1 + Math.max(leftDepth, rightDepth);
}
// TIME: O(n) — visit every node once SPACE: O(h) — call stack depth = tree height
// For balanced tree h = O(log n); skewed tree h = O(n)
```

■ **WHY THIS WORKS: Post-order is natural here: compute left depth, compute right depth, then combine. The recursion elegantly handles all cases — no special handling for leaves needed.**

BFS Level-Order Traversal — Binary Tree

```
// PATTERN: BFS Level-Order (Queue) — O(n) time, O(w) space (w = max width)
// PROBLEM: Return nodes grouped by level
// WHY Queue: BFS explores layer by layer — exactly what we need for level-order
// Process all nodes at depth d before any node at depth d+1
public List<List<Integer>> levelOrder(TreeNode root) {
List<List<Integer>> result = new ArrayList<>();
if (root == null) return result;
// STEP 1: Queue starts with just the root
Queue<TreeNode> queue = new LinkedList<>();
queue.offer(root);
while (!queue.isEmpty()) {
// STEP 2: Snapshot the current level size BEFORE we process it
// WHY: during processing we add children (next level) to queue
```

```
// We must process exactly 'levelSize' nodes for this level
int levelSize = queue.size();
List<Integer> level = new ArrayList<>();
for (int i = 0; i < levelSize; i++) {
TreeNode node = queue.poll();
level.add(node.val);
// STEP 3: Enqueue children for next level
if (node.left != null) queue.offer(node.left);
if (node.right != null) queue.offer(node.right);
}
result.add(level);
}
return result;
}
// TIME: O(n) SPACE: O(w) where w = max width (bottom level can have n/2 nodes)
```

■ **WHY THIS WORKS: Key trick: capture queue.size() BEFORE the inner loop. This is the exact number of nodes at the current level. Adding children during the loop doesn't affect this count.**

DFS with Bounds — Validate BST

```
// PATTERN: DFS with bounds — O(n) time, O(h) space
// PROBLEM: Validate that a binary tree satisfies the BST property
// WHY pass bounds (not just compare to parent): the classic mistake is checking
// only parent — but a right subtree node must be > ALL ancestors on the left path
// EXAMPLE of why naive fails:
// 5
// / \
// 1 4 <- 4 < 5 OK superficially
// / \
// 3 6 <- but 3 < 5 violates BST! (must be > 5 in right subtree of 5)
public boolean isValidBST(TreeNode root) {
return validate(root, Long.MIN_VALUE, Long.MAX_VALUE);
// WHY Long: handles edge case where node.val == Integer.MIN/MAX_VALUE
}
private boolean validate(TreeNode node, long lo, long hi) {
// BASE CASE: null nodes are always valid
if (node == null) return true;
// STEP 1: Current node must be strictly within (lo, hi)
if (node.val <= lo || node.val >= hi) return false;
// STEP 2: Narrow bounds and recurse
// WHY node.val as new bound: all nodes in LEFT subtree must be < node.val
// all nodes in RIGHT subtree must be > node.val
return validate(node.left, lo, node.val) &&
validate(node.right, node.val, hi);
}
// TIME: O(n) SPACE: O(h) — bounds passed on call stack
```

■ **WHY THIS WORKS: Passing (lo, hi) bounds threads the full BST constraint through every recursive call. Without this, you'd only check parent-child relationship, missing transitive violations.**

Post-order DFS — Lowest Common Ancestor

```
// PATTERN: Post-order DFS with global tracking — O(n) time, O(h) space
// PROBLEM: Find the Lowest Common Ancestor of two nodes p and q
```

```
// WHY post-order: we need to know what's in LEFT and RIGHT before deciding
// if current node is the LCA
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
// BASE CASE 1: fell off tree — neither p nor q here
// BASE CASE 2: found p or q — return it immediately
// WHY return early on p or q: if both are in same subtree, the one found
// first will bubble up; if in different subtrees, the split point is LCA
if (root == null || root == p || root == q) return root;
// STEP 1: Search both subtrees
TreeNode leftResult = lowestCommonAncestor(root.left, p, q);
TreeNode rightResult = lowestCommonAncestor(root.right, p, q);
// STEP 2: If both sides returned non-null, p and q are in DIFFERENT subtrees
// WHY: current root is the meeting point — it IS the LCA
if (leftResult != null && rightResult != null) return root;
// STEP 3: Otherwise, LCA is in whichever side returned a non-null value
return leftResult != null ? leftResult : rightResult;
}
// TIME: O(n) SPACE: O(h) call stack
```

Top Tree Interview Questions

Invert Binary Tree

Swap node.left and node.right, then recurse into both. One-liner with ternary

Diameter of Binary Tree

int[] max = {0}. At each node: max[0] = Math.max(max[0], leftH + rightH)

Balanced Binary Tree

Return -1 from helper if unbalanced. Check Math.abs(l-r) > 1

Same Tree

root1==null && root2==null → true. Either null → false. Check val, recurse left, recurse right

Subtree of Another Tree

isSameTree(root, sub) || isSubtree(root.left) || isSubtree(root.right)

LCA of BST

If both p.val and q.val < root.val → go left. Both > root.val → go right. Else root IS LCA

Binary Tree Right Side View

Level order BFS. Take last node of each level

Count Good Nodes

DFS passing maxSoFar. Node is good if node.val >= maxSoFar

Kth Smallest in BST

Iterative inorder with counter. When count==k, that's the answer

Construct BT from Preorder+Inorder

Root = preorder[0]. Find root in inorder — left of it is left subtree

Serialize & Deserialize BT

Preorder DFS with 'N' for null, comma delimiter. Decode with Queue<String>

Binary Tree Max Path Sum

At each node: gain = node.val + max(0,leftGain) + max(0,rightGain). Update global max

6. GRAPHS

Graph Setup — Adjacency List & Grid

```
// GRAPH SETUP — two most common representations
// WHY adjacency list: space-efficient for sparse graphs (most interview problems)
// Space: O(V + E) vs O(V^2) for adjacency matrix
// Option A: List of Lists
int n = 5;
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
graph.get(u).add(v); // directed edge u -> v
graph.get(v).add(u); // undirected: add both directions
// Option B: Weighted graph (store pairs {neighbor, weight})
List<int[]>[] wGraph = new List[n];
for (int i = 0; i < n; i++) wGraph[i] = new ArrayList<>();
wGraph[u].add(new int[]{v, weight});
// Option C: Grid as graph (4-directional)
int[][] DIRS = {{0,1},{0,-1},{1,0},{-1,0}}; // right, left, down, up
// WHY array of pairs: clean way to iterate over 4 neighbors
for (int[] d : DIRS) {
int nr = row + d[0], nc = col + d[1];
if (nr >= 0 && nr < ROWS && nc >= 0 && nc < COLS) {
// valid neighbor at (nr, nc)
}
}
```

DFS — Count Connected Components

```
// PATTERN: DFS + visited array — O(V+E) time, O(V) space
// PROBLEM: Count number of connected components
// WHY DFS: naturally explores all reachable nodes from a starting point
// One DFS call = one connected component
public int countComponents(int n, int[][] edges) {
// STEP 1: Build adjacency list
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
for (int[] e : edges) {
graph.get(e[0]).add(e[1]);
graph.get(e[1]).add(e[0]); // undirected
}
// STEP 2: Track visited nodes to avoid revisiting
// WHY boolean[]: faster than HashSet for integer node IDs
boolean[] visited = new boolean[n];
int components = 0;
for (int i = 0; i < n; i++) {
if (!visited[i]) {
// STEP 3: Start DFS from any unvisited node
// WHY: each unvisited node starts a new component
dfs(graph, visited, i);
components++;
}
}
```

```
return components;
}
private void dfs(List<List<Integer>> graph, boolean[] visited, int node) {
visited[node] = true; // mark before recursing — prevents infinite loop in cycles
for (int neighbor : graph.get(node)) {
if (!visited[neighbor]) {
dfs(graph, visited, neighbor);
}
}
}
// TIME: O(V+E) — each node and edge visited once SPACE: O(V) visited + O(V) stack
```

Union-Find (DSU) — Path Compression + Union by Rank

```
// PATTERN: Union-Find (DSU) — nearly O(1) per operation
// PROBLEM: Detect cycles / find connected components efficiently
// WHY Union-Find: optimized with path compression + union by rank
// Makes find() nearly O(1) amortized — faster than DFS for repeated queries
// KEY IDEA: each component has a "root" representative;
// union merges two components; find returns the root
class UnionFind {
int[] parent; // parent[i] = i's parent in the tree
int[] rank; // rank[i] = approximate depth of subtree at i
UnionFind(int n) {
parent = new int[n];
rank = new int[n];
// STEP 1: Initially each node is its own root
for (int i = 0; i < n; i++) parent[i] = i;
}
// Find root with PATH COMPRESSION
// WHY path compression: flatten the tree so all nodes point directly to root
// makes subsequent find() calls O(1)
int find(int x) {
if (parent[x] != x) {
parent[x] = find(parent[x]); // compress: point directly to root
}
return parent[x];
}
// Union by RANK (merge smaller tree under larger)
// WHY rank: keeps trees shallow — prevents degenerate O(n) chains
boolean union(int x, int y) {
int px = find(x), py = find(y);
if (px == py) return false; // already in same component
if (rank[px] < rank[py]) parent[px] = py;
else if (rank[px] > rank[py]) parent[py] = px;
else { parent[py] = px; rank[px]++; } // equal height: increment one
return true;
}
}
// TIME: O(alpha(n)) per op — alpha is inverse Ackermann, effectively O(1)
// SPACE: O(n) for parent and rank arrays
```

■ **WHY THIS WORKS: Two optimizations make Union-Find nearly O(1): (1) Path compression flattens trees during find(). (2) Union by rank keeps trees balanced. Together they achieve inverse-Ackermann time — practically constant.**

Kahn's BFS — Topological Sort / Course Schedule

```
// PATTERN: BFS Topological Sort (Kahn's Algorithm) — O(V+E) time, O(V) space
// PROBLEM: Return valid ordering of courses given prerequisites (Course Schedule II)
// WHY BFS topo sort: processes nodes in dependency order
// Nodes with no incoming edges (in-degree=0) have no prerequisites — start there
// Also detects cycles: if we can't process all nodes, a cycle exists
public int[] findOrder(int numCourses, int[][] prerequisites) {
// STEP 1: Build graph and compute in-degrees
List<List<Integer>> graph = new ArrayList<>();
int[] inDegree = new int[numCourses];
for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());
for (int[] pre : prerequisites) {
graph.get(pre[1]).add(pre[0]); // pre[1] must come before pre[0]
inDegree[pre[0]]++;
}
// STEP 2: Start BFS from all nodes with no prerequisites (in-degree == 0)
Queue<Integer> queue = new LinkedList<>();
for (int i = 0; i < numCourses; i++) {
if (inDegree[i] == 0) queue.offer(i);
}
int[] order = new int[numCourses];
int idx = 0;
while (!queue.isEmpty()) {
int course = queue.poll();
order[idx++] = course;
// STEP 3: "Remove" this course — reduce in-degree of dependent courses
// WHY: if in-degree reaches 0, all prerequisites satisfied → can take it
for (int next : graph.get(course)) {
if (--inDegree[next] == 0) {
queue.offer(next);
}
}
}
// STEP 4: Cycle check — if we processed all courses, no cycle exists
return idx == numCourses ? order : new int[0];
}
// TIME: O(V+E) SPACE: O(V+E)
```

■ **WHY THIS WORKS: Kahn's insight: nodes with in-degree 0 have no blockers — process them first. As you process each node, reduce neighbors' in-degrees. New zeros are now unblocked. Cycle = some nodes never reach in-degree 0.**

Dijkstra's Algorithm — Network Delay Time

```
// PATTERN: Dijkstra's Shortest Path — O((V+E) log V) time, O(V) space
// PROBLEM: Find shortest time for signal to reach all nodes (Network Delay Time)
// WHY Dijkstra: greedy with min-heap — always processes nearest unvisited node
// Correct for non-negative weights (negative weights need Bellman-Ford)
public int networkDelayTime(int[][] times, int n, int k) {
```

```
// STEP 1: Build weighted adjacency list
List<int[]>[] graph = new List[n + 1];
for (int i = 0; i <= n; i++) graph[i] = new ArrayList<>();
for (int[] t : times) {
graph[t[0]].add(new int[]{t[1], t[2]}); // {destination, weight}
}
// STEP 2: Min-heap ordered by distance: {dist, node}
// WHY min-heap: always greedily processes the closest unvisited node
PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
int[] dist = new int[n + 1];
Arrays.fill(dist, Integer.MAX_VALUE);
dist[k] = 0; // source node: distance 0
pq.offer(new int[]{0, k});
while (!pq.isEmpty()) {
int[] curr = pq.poll();
int d = curr[0];
int node = curr[1];
// WHY skip stale entries: heap may have outdated distances
if (d > dist[node]) continue;
for (int[] neighbor : graph[node]) {
int newDist = dist[node] + neighbor[1];
if (newDist < dist[neighbor[0]]) {
dist[neighbor[0]] = newDist;
pq.offer(new int[]{newDist, neighbor[0]});
}
}
}
int maxDist = 0;
for (int i = 1; i <= n; i++) {
if (dist[i] == Integer.MAX_VALUE) return -1; // unreachable node
maxDist = Math.max(maxDist, dist[i]);
}
return maxDist;
}
// TIME: O((V+E) log V) SPACE: O(V+E)
```

Top Graph Interview Questions

Number of Islands (Grid DFS)

DFS from each '1', mark all connected land as '0'. Count DFS calls

Clone Graph

HashMap<Node,Node>. DFS: if in map return clone; else create clone, add to map, clone neighbors

Max Area of Island

DFS returns 1 + sum of 4 neighbor DFS calls. Track global max

Pacific Atlantic Water Flow

BFS from Pacific borders (top+left) and Atlantic (bottom+right). Return intersection

Word Ladder (BFS shortest path)

BFS. For each word, try all 26 substitutions. If in wordSet, add to queue. Count levels

Graph Valid Tree

Must have exactly n-1 edges AND be connected (no cycle). Use Union-Find or DFS

Accounts Merge

Union-Find on emails. Group emails by root. Map root→owner for final result

Swim in Rising Water

Binary search on answer time + BFS feasibility, OR Dijkstra treating time as weight

Cheapest Flights Within K Stops

Modified Bellman-Ford: relax exactly k+1 times. Copy dist[] each iteration

Bipartite Check

BFS/DFS 2-coloring. If adjacent nodes have same color → not bipartite

7. HASH MAPS & HASH SETS

HashMap — Two Sum

```
// PATTERN: HashMap Two-Pass — O(n) time, O(n) space
// PROBLEM: Two Sum — find indices of two numbers that add to target
// WHY HashMap: for each element x, we need O(1) lookup of "has target-x been seen?"
// Brute force is O(n^2) with nested loops
public int[] twoSum(int[] nums, int target) {
// STEP 1: Map stores value -> index
// WHY value as key: we search by VALUE (target - current), retrieve INDEX
Map<Integer, Integer> valueToIndex = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
int complement = target - nums[i];
// STEP 2: Check if complement already exists in map
// WHY check BEFORE inserting: avoids using same index twice (e.g. target=6, num=3)
if (valueToIndex.containsKey(complement)) {
return new int[]{valueToIndex.get(complement), i};
}
// STEP 3: Record this number's index for future lookups
valueToIndex.put(nums[i], i);
}
return new int[]{-1, -1}; // no solution found
}
// TIME: O(n) single pass SPACE: O(n) for the map
```

■ WHY THIS WORKS: HashMap converts 'have I seen X before?' from O(n) scan to O(1) lookup. We check complement BEFORE inserting to avoid using the same element twice.

HashSet — Longest Consecutive Sequence

```
// PATTERN: HashSet for O(1) membership + smart chain start — O(n) time, O(n) space
// PROBLEM: Find length of longest consecutive sequence (unsorted array allowed)
// WHY HashSet: O(1) lookup lets us check neighbors without sorting (which would be O(n log n))
// KEY INSIGHT: only START a chain when num-1 does NOT exist
// This ensures each chain is counted exactly once
public int longestConsecutive(int[] nums) {
// STEP 1: Load all numbers into a set for O(1) lookup
Set<Integer> numSet = new HashSet<>();
for (int n : nums) numSet.add(n);
int longest = 0;
for (int n : numSet) {
// STEP 2: Only start counting from a sequence's beginning
// WHY check !numSet.contains(n-1): if n-1 exists, n is NOT the start
// skip it — we'll count it when we process the actual start
// Without this, we'd recount every subsequence: O(n) per element = O(n^2) total
if (!numSet.contains(n - 1)) {
int currentNum = n;
int currentLength = 1;
// STEP 3: Extend chain as long as consecutive numbers exist
while (numSet.contains(currentNum + 1)) {
currentNum++;
currentLength++;
```

```
}
longest = Math.max(longest, currentLength);
}
}
return longest;
}
// TIME: O(n) — each number checked at most twice (once outer, once inner)
// SPACE: O(n) for the HashSet
```

■ WHY THIS WORKS: The 'start of sequence' check is the key optimization. Without it, we'd extend chains from every element: O(n²). With it, each element participates in exactly one chain extension: O(n).

HashMap + PriorityQueue — Top K Frequent Elements

```
// PATTERN: Prefix Sum + HashMap — O(n) time, O(n) space
// PROBLEM: Find if array can be split into k consecutive number groups of size groupSize
// GENERAL VERSION: Count subarrays with exact property using cumulative state
// SIMPLER: Top K Frequent Elements — HashMap + PriorityQueue
public int[] topKFrequent(int[] nums, int k) {
// STEP 1: Count frequency of each number
// WHY HashMap: arbitrary integer keys, O(1) increment with merge()
Map<Integer, Integer> count = new HashMap<>();
for (int n : nums) count.merge(n, 1, Integer::sum);
// STEP 2: Min-heap of size k ordered by frequency
// WHY min-heap of size k (not max-heap): we only keep top k
// When heap exceeds k, we remove the LEAST frequent (min) — correct!
// Max-heap would require sorting everything first: O(n log n)
PriorityQueue<Integer> minHeap =
new PriorityQueue<>((a, b) -> count.get(a) - count.get(b));
for (int key : count.keySet()) {
minHeap.offer(key);
// STEP 3: Keep heap size exactly k by removing the least frequent
if (minHeap.size() > k) minHeap.poll();
}
// STEP 4: Extract results (min-heap, so build result in reverse)
int[] result = new int[k];
for (int i = k - 1; i >= 0; i--) result[i] = minHeap.poll();
return result;
}
// TIME: O(n log k) SPACE: O(n) for map + O(k) for heap
```

Top HashMap/HashSet Interview Questions

Valid Anagram

int[26] count array. Increment for s, decrement for t. Check all zeros

Group Anagrams

Sort each word's chars as key. computeIfAbsent(key, k->new ArrayList<>()).add(word)

Contains Duplicate

HashSet: return !set.add(num). add() returns false if already present

First Unique Character

Two-pass: int[26] frequency, then find first with count==1

Intersection of Two Arrays

Set from arr1. For arr2, if set.contains → add to result set

Happy Number

HashSet of seen sums. If sum==1 → true. If seen before → cycle → false

Isomorphic Strings

Two maps: s-char→t-char and t-char→s-char. Check bidirectional consistency

4Sum II

Two-pass: count all (a+b) in map. For each (c+d), count map[-(c+d)]

Subarray Sum Equals K

Prefix sum + HashMap<Integer,Integer> freq. count+=freq.get(prefix-k)

Minimum Window Substring

need map + have counter. When have==need, shrink from left. Track min window

8. HEAPS (PRIORITY QUEUES)

Min-Heap of Size K — Kth Largest Element

```
// PATTERN: Min-Heap of size K — O(n log k) time, O(k) space
// PROBLEM: Find the Kth largest element in an array
// WHY min-heap of size k: root of min-heap = smallest of top-k
// = Kth largest overall. Heap size never exceeds k → space efficient.
// WHY NOT sort: O(n log n) vs O(n log k). For small k, log k << log n
public int findKthLargest(int[] nums, int k) {
// STEP 1: Min-heap stores the k largest elements seen so far
// WHY min-heap: root is the smallest of our k candidates
// When a new element beats the root, it replaces it
PriorityQueue<Integer> minHeap = new PriorityQueue<>();
for (int n : nums) {
minHeap.offer(n); // always add the current element
// STEP 2: If heap grows beyond k, remove the smallest
// WHY: we only care about top k. Removing the minimum preserves the k largest
if (minHeap.size() > k) {
minHeap.poll(); // evict the current minimum (weakest of top-k)
}
}
// STEP 3: Root of min-heap = smallest of the k largest = Kth largest
return minHeap.peek();
}
// TIME: O(n log k) — n insertions each costing O(log k)
// SPACE: O(k) — heap never exceeds size k
```

■ **WHY THIS WORKS: Counter-intuitive: to find Kth LARGEST, we use a MIN-heap. The heap maintains the top-k elements. Its root (minimum of top-k) is exactly the Kth largest. When we see a bigger element, we swap out the current minimum.**

Two Heaps — Find Median from Data Stream

```
// PATTERN: Two Heaps (Max + Min) — O(log n) add, O(1) median
// PROBLEM: Find median from a stream of numbers at any time
// WHY two heaps: we need the MIDDLE element dynamically
// Split data into lower half (max-heap) and upper half (min-heap)
// Median is always at the boundary between the two halves
class MedianFinder {
// lo = max-heap holding the LOWER half of numbers
// WHY max-heap: we want quick access to the LARGEST of the lower half (left middle)
PriorityQueue<Integer> lo = new PriorityQueue<>(Collections.reverseOrder());
// hi = min-heap holding the UPPER half of numbers
// WHY min-heap: we want quick access to the SMALLEST of the upper half (right middle)
PriorityQueue<Integer> hi = new PriorityQueue<>();
public void addNum(int num) {
// STEP 1: Route all numbers through lo first (always add to lo)
lo.offer(num);
// STEP 2: Balance: ensure lo's max <= hi's min
// WHY: lo.peek() might be > hi.peek() after adding num
// If so, move lo's max to hi
if (!hi.isEmpty() && lo.peek() > hi.peek()) {
```

```
hi.offer(lo.poll());
}
// STEP 3: Rebalance sizes — lo is allowed to have 1 extra element
// WHY: ensures lo.size() == hi.size() or lo.size() == hi.size() + 1
if (lo.size() > hi.size() + 1) hi.offer(lo.poll());
if (hi.size() > lo.size()) lo.offer(hi.poll());
}
public double findMedian() {
// WHY lo has the median when sizes differ: lo is the "extra" element
if (lo.size() > hi.size()) return lo.peek();
// Even total: median is average of two middle elements
return (lo.peek() + hi.peek()) / 2.0;
}
}
// add: O(log n) findMedian: O(1) Space: O(n)
```

■ **WHY THIS WORKS: Two heaps partition data at the median. lo (max-heap) = lower half; hi (min-heap) = upper half. Median is always at their boundary. Balancing invariant: lo.size() == hi.size() or lo.size() == hi.size()+1.**

Min-Heap K-Way Merge — Merge K Sorted Lists

```
// PATTERN: Min-Heap for K-way merge — O(n log k) time, O(k) space
// PROBLEM: Merge k sorted linked lists into one sorted list
// WHY min-heap: when merging k sorted lists, we need the global minimum
// at each step. Heap gives us that in O(log k) instead of O(k) linear scan
public ListNode mergeKLists(ListNode[] lists) {
// STEP 1: Custom min-heap comparing ListNodes by their val
PriorityQueue<ListNode> minHeap =
new PriorityQueue<>((a, b) -> a.val - b.val);
// STEP 2: Seed heap with the HEAD of each list
// WHY only heads: smallest elements come first; we'll add successors as we go
for (ListNode head : lists) {
if (head != null) minHeap.offer(head);
}
ListNode dummy = new ListNode(0); // WHY dummy: avoids null check for first node
ListNode curr = dummy;
while (!minHeap.isEmpty()) {
// STEP 3: Extract globally smallest node
ListNode smallest = minHeap.poll();
curr.next = smallest;
curr = curr.next;
// STEP 4: Advance that list — push its next node into heap
// WHY: heap must always contain the FRONT of each unfinished list
if (smallest.next != null) {
minHeap.offer(smallest.next);
}
}
return dummy.next;
}
// TIME: O(n log k) where n=total nodes, k=number of lists
// SPACE: O(k) for the heap (at most k nodes at a time)
```

Top Heap Interview Questions

Top K Frequent Elements

HashMap for counts + min-heap of size k keyed by count. O(n log k)

K Closest Points to Origin

Max-heap of size k on distance. Evict farthest when size exceeds k

Task Scheduler (Idle time)

Max-heap of task frequencies. Simulate CPU cycles. Idle when heap empty mid-round

Kth Smallest in Sorted Matrix

Min-heap BFS from (0,0). PQ stores {val, row, col}. Poll k times

Ugly Number II

Three variables: i2, i3, i5. Next ugly = min(uglies[i2]*2, uglies[i3]*3, uglies[i5]*5)

IPO (Two-heap greedy)

Min-heap for capital; max-heap for profit. For each round, unlock available projects and pick max profit

9. TRIES (PREFIX TREES)

Full Trie Implementation — Insert, Search, StartsWith

```
// PATTERN: Trie (Prefix Tree) — O(m) per operation where m = word length
// WHY Trie over HashMap: Trie shares prefixes among words saving memory
// and enables O(m) prefix search — HashMap would need full key match
// WHY TrieNode[26]: faster than HashMap<Character,TrieNode> for lowercase letters
// Fixed-size array avoids hashing overhead
class Trie {
// ■■■ INNER NODE CLASS ■■■
```

```
// SPACE: O(ALPHABET * N * M) where N=number of words, M=avg word length
```

■ WHY THIS WORKS: Trie shines when many words share prefixes. 'cat', 'car', 'card' share 'ca' — stored once. Search is O(m) regardless of how many words are stored. HashMap would need a separate key for each word.

Trie + Backtracking — Word Search II

```
// PATTERN: Trie + DFS Backtracking — O(m * 4^m) where m = word length
// PROBLEM: Word Search II — find all words from a list that exist in the board
// WHY Trie over checking each word separately:
// Searching board for W words individually = O(W * m * 4^m)
// Trie prunes paths that can't lead to any word = much faster in practice
class WordSearch2 {
public List<String> findWords(char[][] board, String[] words) {
// STEP 1: Build Trie from all target words
// WHY Trie: during board DFS, we navigate Trie simultaneously
// If current path isn't a Trie prefix, prune entire branch
Trie trie = new Trie();
for (String w : words) trie.insert(w);
Set<String> result = new HashSet<>(); // WHY Set: avoid duplicate results
int ROWS = board.length, COLS = board[0].length;
// STEP 2: Start DFS from every cell as potential word start
for (int r = 0; r < ROWS; r++) {
for (int c = 0; c < COLS; c++) {
dfs(board, trie.root, r, c, new StringBuilder(), result);
}
}
return new ArrayList<>(result);
}
private void dfs(char[][] board, Trie.TrieNode node, int r, int c,
StringBuilder path, Set<String> result) {
// Boundary or visited-cell check
if (r < 0 || r >= board.length || c < 0 || c >= board[0].length
|| board[r][c] == '#') return;
char ch = board[r][c];
int idx = ch - 'a';
// STEP 3: Prune — if no Trie child for this char, stop this path
// WHY: current path can't lead to any target word
if (node.children[idx] == null) return;
Trie.TrieNode next = node.children[idx];
path.append(ch);
// STEP 4: If this node marks a word end, collect it
if (next.isEnd) result.add(path.toString());
// STEP 5: Mark cell as visited, explore 4 neighbors, then restore
board[r][c] = '#'; // WHY #: prevents revisiting same cell in this path
dfs(board, next, r+1, c, path, result);
dfs(board, next, r-1, c, path, result);
dfs(board, next, r, c+1, path, result);
dfs(board, next, r, c-1, path, result);
board[r][c] = ch; // RESTORE: backtrack (unmark visited)
path.deleteCharAt(path.length() - 1); // backtrack path string
}
}
```

Top Trie Interview Questions

Implement Trie

TrieNode with TrieNode[26] children and boolean isEnd. Insert/Search/StartsWith each O(m)

Design Add and Search Words

Trie + DFS for '.' wildcard. On '.', try all 26 children recursively

Replace Words

Insert all roots. For each word, walk Trie — stop at first isEnd node (shortest root)

Map Sum Pairs

TrieNode stores int val. prefix sum = DFS summing all vals in subtree

Longest Word in Dictionary

Insert all words. BFS/DFS — only traverse nodes where isEnd==true

Autocomplete System

Trie + frequency HashMap per TrieNode. On '#', insert full sentence. On char, DFS for top-3

10. BINARY SEARCH

Binary Search Templates — Exact, Left Bound, Right Bound

```
// PATTERN: Classic Binary Search — O(log n) time, O(1) space
// WHY Binary Search: on a sorted/monotone space, we can eliminate HALF the candidates
// each step. log2(10^9) = ~30 steps — incredibly fast
// TEMPLATE 1: Find exact value
public int binarySearch(int[] arr, int target) {
int lo = 0, hi = arr.length - 1;
while (lo <= hi) {
// WHY lo + (hi-lo)/2 instead of (lo+hi)/2:
// (lo+hi) can OVERFLOW for large values! Safe formula avoids it.
int mid = lo + (hi - lo) / 2;
if (arr[mid] == target) return mid; // found
else if (arr[mid] < target) lo = mid + 1; // target in right half
else hi = mid - 1; // target in left half
}
return -1; // not found
}
// TEMPLATE 2: Find LEFTMOST position satisfying a condition (lower_bound)
// WHY needed: when there are duplicates and we want first occurrence
public int leftBound(int[] arr, int target) {
int lo = 0, hi = arr.length; // WHY hi = length (not length-1): allows insertion at end
while (lo < hi) { // WHY lo < hi (not lo <= hi): we want lo == hi at end
int mid = lo + (hi - lo) / 2;
if (arr[mid] < target) lo = mid + 1; // target is to the right
else hi = mid; // mid could be answer, keep it
}
return lo; // lo == hi == first position where arr[pos] >= target
}
// TEMPLATE 3: Find RIGHTMOST position (upper_bound)
public int rightBound(int[] arr, int target) {
int lo = 0, hi = arr.length;
while (lo < hi) {
int mid = lo + (hi - lo) / 2;
if (arr[mid] <= target) lo = mid + 1; // mid could be answer but check right
else hi = mid;
}
return lo - 1; // lo-1 = last position where arr[pos] <= target
}
```

■ WHY THIS WORKS: Three templates cover all cases. Template 1: exact match. Template 2: first occurrence (lo < hi, hi=mid). Template 3: last occurrence (lo <= mid moves lo past). Safe mid formula prevents integer overflow.

Binary Search on Answer — Koko Eating Bananas

```
// PATTERN: Binary Search on Answer — O(n log(max-min)) time
// PROBLEM: Find minimum eating speed for Koko to finish all bananas in h hours
// WHY search on answer: we don't search in the array — we search in the ANSWER SPACE
// The answer (speed) is monotone: if speed k works, any speed > k also works
// This monotonicity lets us binary search on k directly
public int minEatingSpeed(int[] piles, int h) {
```

```
// STEP 1: Define search space
// WHY 1 to max(piles): min possible speed=1 (eat 1/hr), max=largest pile (finish in 1hr)
int lo = 1;
int hi = Arrays.stream(piles).max().getAsInt();
while (lo < hi) {
int mid = lo + (hi - lo) / 2; // candidate speed
// STEP 2: Check if speed 'mid' can finish in h hours
// WHY Math.ceil(pile/speed) = (pile + speed - 1) / speed in integer arithmetic
if (canFinish(piles, mid, h)) {
hi = mid; // mid works, but maybe something smaller also works → try left
} else {
lo = mid + 1; // mid too slow, must go faster
}
}
return lo; // minimum speed that works
}
private boolean canFinish(int[] piles, int speed, int h) {
int hours = 0;
for (int pile : piles) {
// WHY ceiling division: partial hours round up (eat pile in ceil(pile/speed) hours)
hours += (pile + speed - 1) / speed;
}
return hours <= h; // can we finish within h hours at this speed?
}
// TIME: O(n log(max_pile)) SPACE: O(1)
```

■ WHY THIS WORKS: 'Search on answer' pattern: instead of searching in data, we search in the answer range. Key requirement: the answer space must be MONOTONE (if k works, k+1 also works). Then binary search finds the boundary.

Binary Search on Rotated Array

```
// PATTERN: Binary Search on Rotated Array — O(log n) time, O(1) space
// PROBLEM: Search for target in a sorted array that was rotated at some pivot
// WHY Binary Search still works: ONE of the two halves is ALWAYS sorted
// We identify the sorted half and check if target falls in it
public int search(int[] nums, int target) {
int lo = 0, hi = nums.length - 1;
while (lo <= hi) {
int mid = lo + (hi - lo) / 2;
if (nums[mid] == target) return mid;
// STEP 1: Determine which half is sorted
// WHY: in any rotation, one half is always sorted
if (nums[lo] <= nums[mid]) {
// LEFT half [lo..mid] is sorted
// STEP 2a: Is target in the sorted left half?
if (nums[lo] <= target && target < nums[mid]) {
hi = mid - 1; // target in left half
} else {
lo = mid + 1; // target in right half
}
} else {
// RIGHT half [mid..hi] is sorted
```

```
// STEP 2b: Is target in the sorted right half?
if (nums[mid] < target && target <= nums[hi]) {
lo = mid + 1; // target in right half
} else {
hi = mid - 1; // target in left half
}
}
}
return -1;
}
// TIME: O(log n) SPACE: O(1)
```

Top Binary Search Interview Questions

Search a 2D Matrix

Treat as 1D sorted array. row = mid/cols, col = mid%cols

Find Minimum in Rotated Sorted Array

If nums[mid] > nums[right], min is in right half. Else left half

Time Based Key-Value Store

TreeMap<Integer,String> per key. get() uses floorEntry(timestamp)

Capacity to Ship in D Days

Binary search on capacity [maxWeight, sumWeight]. canShip() checks days needed

Find Peak Element

Binary search: if nums[mid] < nums[mid+1], peak is to the right; else left

Median of Two Sorted Arrays

Binary search partition on smaller array. Hard but O(log(min(m,n)))

Split Array Largest Sum

Binary search on max subarray sum. canSplit() checks if k subarrays fit

LIS in O(n log n)

Maintain tails[] array. Binary search (leftBound) for each element's placement

11. DYNAMIC PROGRAMMING

Framework: State → Transition → Base Case → Optimize Space

- 1. Define STATE: what variables completely describe a subproblem?
- 2. Define TRANSITION: dp[state] = f(dp[smaller states])
- 3. Define BASE CASE: smallest/trivial subproblems with known answers
- 4. OPTIMIZE: can you reduce 2D dp to 1D? 1D to O(1)?

1D DP + Space Optimization — Climbing Stairs

```
// PATTERN: 1D DP with O(1) Space Optimization — O(n) time
// PROBLEM: Climbing stairs — how many distinct ways to reach step n (1 or 2 steps)?
// WHY DP: overlapping subproblems — ways(n) = ways(n-1) + ways(n-2) (Fibonacci pattern)
// WHY O(1) space: dp[i] only depends on dp[i-1] and dp[i-2] — keep only last two
public int climbStairs(int n) {
if (n <= 2) return n; // base cases: 1 way for n=1, 2 ways for n=2
// STEP 1: Instead of int[] dp array, use two variables
// WHY: dp[i] = dp[i-1] + dp[i-2] → only need previous two values
int prev2 = 1; // dp[1] = 1 way to reach step 1
int prev1 = 2; // dp[2] = 2 ways to reach step 2
for (int i = 3; i <= n; i++) {
// STEP 2: Current = sum of two previous (Fibonacci recurrence)
int curr = prev1 + prev2;
prev2 = prev1; // slide window forward
prev1 = curr;
}
return prev1;
}
// TIME: O(n) SPACE: O(1) — reduced from O(n) array to 2 variables
```

Unbounded Knapsack DP — Coin Change

```
// PATTERN: Bottom-Up DP (Unbounded Knapsack) — O(amount * coins) time, O(amount) space
// PROBLEM: Coin Change — minimum coins to make exact amount
// WHY DP: optimal substructure — min coins for amount a = 1 + min coins for (a - coin)
// for all coins that fit. Subproblems overlap massively — DP avoids recomputation
public int coinChange(int[] coins, int amount) {
// STEP 1: dp[a] = minimum number of coins to make amount a
// WHY amount+1 as infinity: impossible to need more than amount coins (all 1s)
int[] dp = new int[amount + 1];
Arrays.fill(dp, amount + 1); // initialize to "infinity"
dp[0] = 0; // WHY dp[0]=0: 0 coins needed to make amount 0 (base case)
// STEP 2: Build solution for each amount from 1 to amount
for (int a = 1; a <= amount; a++) {
// STEP 3: Try every coin denomination
for (int coin : coins) {
// WHY coin <= a: can only use coin if it doesn't exceed current amount
if (coin <= a) {
// TRANSITION: dp[a] = min(dp[a], 1 + dp[a - coin])
// WHY 1 + dp[a-coin]: use one coin, solve remaining (a-coin)
dp[a] = Math.min(dp[a], 1 + dp[a - coin]);
}
```

```
}
}
// STEP 4: If dp[amount] still > amount, no solution exists
return dp[amount] > amount ? -1 : dp[amount];
}
// TIME: O(amount * |coins|) SPACE: O(amount)
```

■ WHY THIS WORKS: Unbounded knapsack: each coin can be used unlimited times. Bottom-up builds answers for smaller amounts first. dp[a] depends on dp[a-coin] which is already computed when we process amount a.

2D DP — Longest Common Subsequence

```
// PATTERN: 2D DP for two sequences — O(m*n) time, O(m*n) or O(n) space
// PROBLEM: Longest Common Subsequence of two strings
// WHY 2D DP: need to track state for BOTH strings simultaneously
// dp[i][j] = LCS length of s1[0..i-1] and s2[0..j-1]
public int longestCommonSubsequence(String s1, String s2) {
int m = s1.length(), n = s2.length();
// STEP 1: dp table — extra row/col of zeros = base case (empty string LCS = 0)
int[][] dp = new int[m + 1][n + 1];
for (int i = 1; i <= m; i++) {
for (int j = 1; j <= n; j++) {
// STEP 2: Two cases based on whether current chars match
if (s1.charAt(i - 1) == s2.charAt(j - 1)) {
// CASE A: chars match — extend LCS by 1
// WHY dp[i-1][j-1]: LCS of strings WITHOUT these matching chars, plus 1
dp[i][j] = 1 + dp[i - 1][j - 1];
} else {
// CASE B: chars don't match — take best of skipping one char from either
// WHY max: we try removing last char of s1 OR last char of s2
dp[i][j] = Math.max(dp[i - 1][j], // skip last char of s1
dp[i][j - 1]); // skip last char of s2
}
}
}
return dp[m][n];
}
// TIME: O(m*n) SPACE: O(m*n) — can optimize to O(n) using rolling array
```

0/1 Knapsack DP — Partition Equal Subset Sum

```
// PATTERN: 0/1 Knapsack DP — O(n * capacity) time, O(capacity) space
// PROBLEM: Partition array into two equal-sum subsets
// WHY Knapsack: each number is either IN or NOT IN the subset (0/1 choice)
// dp[w] = can we form sum w using numbers seen so far?
public boolean canPartition(int[] nums) {
int total = Arrays.stream(nums).sum();
// STEP 1: Odd sum → impossible to split into equal halves
if (total % 2 != 0) return false;
int target = total / 2;
// STEP 2: boolean dp[w] = "can we form sum w from some subset?"
boolean[] dp = new boolean[target + 1];
dp[0] = true; // WHY dp[0]=true: empty subset forms sum 0 (always possible)
for (int num : nums) {
```

```
// STEP 3: CRITICAL — iterate RIGHT to LEFT to prevent reusing same element
// WHY right to left: ensures we use each num at most ONCE (0/1 knapsack)
// Left to right would allow a number to be used multiple times
for (int w = target; w >= num; w--) {
// TRANSITION: can we form sum w by EITHER not taking num (dp[w])
// OR taking num and forming (w - num) with remaining?
dp[w] = dp[w] || dp[w - num];
}
}
return dp[target];
}
// TIME: O(n * target) SPACE: O(target) — reduced from O(n * target) 2D
```

■ WHY THIS WORKS: Iterating RIGHT to LEFT is the crucial optimization for 0/1 Knapsack with a 1D array. It ensures dp[w-num] refers to the state BEFORE processing current num — preventing using it twice.

Word Break DP

```
// PATTERN: Word Break DP — O(n^2 * L) time where L = avg word length
// PROBLEM: Can string s be segmented into words from wordDict?
// WHY DP: overlapping subproblems — same prefix validated multiple times in recursion
// WHY HashSet: O(1) word lookup vs O(L) list contains
public boolean wordBreak(String s, List<String> wordDict) {
// STEP 1: HashSet for O(1) lookup
Set<String> wordSet = new HashSet<>(wordDict);
// STEP 2: dp[i] = "can s[0..i-1] be formed from dictionary words?"
boolean[] dp = new boolean[s.length() + 1];
dp[0] = true; // WHY true: empty string is trivially segmentable (base case)
for (int i = 1; i <= s.length(); i++) {
// STEP 3: Try every possible last word ending at position i
// WHY j from 0 to i: check if s[j..i-1] is in dict AND s[0..j-1] was valid
for (int j = 0; j < i; j++) {
if (dp[j] && wordSet.contains(s.substring(j, i))) {
dp[i] = true;
break; // WHY break: found one valid segmentation — no need to continue
}
}
}
return dp[s.length()];
}
// TIME: O(n^2 * L) where L = substring comparison cost SPACE: O(n + dict size)
```

Top DP Interview Questions

House Robber

dp[i] = Math.max(dp[i-1], dp[i-2]+nums[i]). Optimize to O(1): prev2, prev1 variables

House Robber II (circular)

Run rob() twice: on nums[0..n-2] and nums[1..n-1]. Return max

Longest Increasing Subsequence O(n log n)

tails[] array + binary search. Replace first element >= current. tails.size() = LIS length

Edit Distance

dp[i][j] = min(insert, delete, replace). If chars match, dp[i][j]=dp[i-1][j-1]

Unique Paths

dp[i][j] = dp[i-1][j] + dp[i][j-1]. Optimize to 1D: dp[j] += dp[j-1]

Combination Sum IV (order matters)

dp[target] += dp[target-num] for each target. Different from knapsack — order matters

Decode Ways

dp[i] = ways to decode s[0..i-1]. Check single digit and two-digit validity

Best Time Stock with Cooldown

States: HELD, SOLD, REST. Transition between states at each price

Longest Palindromic Substring

Expand from center (2 cases). Track best start + maxLen. O(n^2) time O(1) space

Regular Expression Matching

dp[i][j] = s[0..i-1] matches p[0..j-1]. Handle '.' and '*' transitions carefully

Burst Balloons (Interval DP)

dp[i][j] = max coins from balloons i..j. Try each as LAST burst: balloon*left*right

Jump Game II (min jumps)

Greedy: curEnd = farthest reachable now; farthest = max reachable from here. Jump at curEnd

12. CHEAT SHEET & INTERVIEW STRATEGY

Big-O Reference

Complexity	Name	Typical Use Cases	n=10^6 Result
O(1)	Constant	HashMap get, array[i]	~1 op — instant
O(log n)	Logarithmic	Binary search, heap ops	~20 ops — fast
O(n)	Linear	Single loop, BFS/DFS	10^6 — OK
O(n log n)	Linearithmic	Merge sort, heap sort	~2x10^7 — OK
O(n^2)	Quadratic	Nested loops, bubble sort	10^12 — TOO SLOW
O(2^n)	Exponential	Subsets, naive recursion	HUGE — avoid
O(n!)	Factorial	Permutations	MASSIVE — avoid

Problem Signal → Pattern Map

When You See...	Use This Pattern	Java Collection
Find pair/triplet summing to X (sorted)	Two Pointers	int lo, hi
Subarray with max/min of size k	Sliding Window	int l, r
Range queries on array	Prefix Sum	int[] prefix
Sorted array, find target	Binary Search	lo + (hi-lo)/2
Tree paths, depth, diameter	DFS (recursion)	recursive helper
Level-by-level, shortest path	BFS	Queue (LinkedList)
All combinations, permutations	Backtracking	boolean[] used
Optimal with overlapping subproblems	Dynamic Programming	int[] or int[][] dp
Connected components, cycle detect	Union-Find / DFS	UnionFind class
Top K elements	Heap	PriorityQueue<Integer>
Next greater/smaller	Monotonic Stack	Deque<Integer> stack
Prefix string search / autocomplete	Trie	TrieNode[26]
Dependency ordering	Topological Sort (Kahn's)	Queue + inDegree[]
Frequency counting	HashMap	map.merge(k,1,Integer::sum)
Duplicate check, membership	HashSet	set.add() / set.contains()

Java One-Liners Worth Memorizing

Java Interview One-Liners

```
// Safe binary search mid (prevents integer overflow)
int mid = lo + (hi - lo) / 2;
// Integer ceiling division (avoids Math.ceil and double cast)
int ceil = (dividend + divisor - 1) / divisor;
```

```
// Swap two elements in array
int tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
// Increment map count safely (no null check needed)
map.merge(key, 1, Integer::sum);
map.put(key, map.getOrDefault(key, 0) + 1); // equivalent
// Group into map (create list on first encounter)
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value);
// Reverse a portion of array in-place (e.g., for rotate array)
for (int lo = 0, hi = end; lo < hi; lo++, hi--) {
int tmp = arr[lo]; arr[lo] = arr[hi]; arr[hi] = tmp;
}
// Sort 2D array by first element, break ties by second (ascending)
Arrays.sort(intervals, (a, b) -> a[0] != b[0] ? a[0]-b[0] : a[1]-b[1]);
// Sort array of strings by length, then alphabetically
Arrays.sort(words, (a, b) -> a.length() != b.length() ?
a.length()-b.length() : a.compareTo(b));
// Char to index mapping (lowercase only)
int idx = c - 'a'; // 'a'->0, 'b'->1, ..., 'z'->25
char c = (char)('a' + idx); // reverse
// Long multiplication to avoid int overflow
long product = (long) a * b; // cast BEFORE multiplication
// Integer bounds for comparison init
int best = Integer.MIN_VALUE; // for finding maximum
int best = Integer.MAX_VALUE; // for finding minimum
// Check if number is power of 2
boolean isPow2 = (n > 0) && ((n & (n-1)) == 0); // WHY: powers of 2 have single set bit
```

Interview Problem-Solving Steps

1. CLARIFY: Ask about constraints (n size, negative numbers, duplicates, null input). Confirm return type. Understand edge cases.

2. EXAMPLES: Walk through 2-3 examples manually. Include edge cases: empty input, single element, all same.

3. BRUTE FORCE: State the naive O(n²) or O(n!) solution first. Explain why it's too slow. Then optimize.

4. PATTERN MATCH: Identify which DS/algorithm pattern applies. Explain your reasoning.

5. CODE: Write clean Java. Use descriptive variable names. Handle edge cases. Add brief inline comments.

6. COMPLEXITY: State Time and Space complexity. Justify each.

7. TEST: Trace through your code with the given example. Then test 1-2 edge cases mentally.

Top 25 Must-Solve Problems

- Two Sum | Maximum Subarray (Kadane's)
- Longest Substring Without Repeating Chars | Valid Parentheses
- Reverse Linked List | Linked List Cycle (Floyd's)
- Merge Two Sorted Lists | LRU Cache (LinkedHashMap)
- Invert Binary Tree | Max Depth of Binary Tree
- Validate BST | Lowest Common Ancestor
- Binary Tree Level Order (BFS) | Max Path Sum
- Number of Islands | Course Schedule (Topological Sort)

■ Binary Search | Search in Rotated Array
■ Top K Frequent Elements | Find Median from Data Stream
■ Climbing Stairs | House Robber | Coin Change
■ Longest Common Subsequence | Word Break
■ 3Sum | Container With Most Water
■ Merge Intervals | Non-overlapping Intervals
■ Implement Trie | Word Search II

■ TIP: Consistency beats intensity. Solve 2-3 LeetCode problems DAILY. After each problem: optimize → test edge cases → analyze O() complexity → understand the PATTERN not just the solution. Master 12 patterns and you can solve 90% of interview questions. Good luck!